

“DRIVECARE PRO – AUTOMOTIVE MOBILE APP FOR VEHICLE OWNER SELF-SERVICE”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

P. BHAVAN :2520090146

M. SHIVA :2520090135

N. SAINATHREDDY :2520090115

Under the Esteemed Guidance of

Dr. Khaja Shareef

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 781 5026916 www.kluh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Project Report entitled **“DriveCare PRO – Automotive Mobile App for Vehicle Owner Self-Service”** is a record of bonafide work carried out by **P. Bhavan – 2520090146, M. Shiva – 2520090135, N. Sainathreddy – 2520090115** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Information Technology at K L (Deemed to be) University. The work described in this report is based on the submitted DriveCare PRO project implementation.

The results and technical descriptions embodied in this report have been prepared for the academic project and have been organized according to the prescribed project-documentation structure. The implementation described includes the frontend, Node.js/Express backend, MongoDB/Mongoose database layer, REST APIs, authentication middleware and application modules present in the project.

P. Bhavan – 2520090146

M. Shiva – 2520090135

N. Sainathreddy – 2520090115

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **“DriveCare PRO – Automotive Mobile App for Vehicle Owner Self-Service”** is a bonafide work done and submitted by **P. Bhavan – 2520090146, M. Shiva – 2520090135 and N. Sainathreddy – 2520090115** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Information Technology, K L (Deemed to be) University, during the academic year **2026–2027**.

The project has been carried out under the guidance of Dr. Khaja Shareef and demonstrates the application of web development, REST API design, database management, authentication and software engineering principles.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely guidance, encouragement and support provided by many people. We express our sincere gratitude to our project guide, **Dr. Khaja Shareef**, Department of Computer Science and Information Technology, for his valuable guidance, technical suggestions and continuous support throughout the development of DriveCare PRO.

We express our gratitude to the Department of Computer Science Engineering and Department of Computer Science and Information Technology for providing the academic environment and facilities required to complete this project-based work. We also thank the teaching and non-teaching staff members who supported us directly or indirectly during the development and documentation process.

Finally, we thank our parents, friends and classmates for their encouragement and cooperation during the completion of this project.

P. Bhavan – 2520090146

M. Shiva – 2520090135

N. Sainathreddy – 2520090115

Abstract

Vehicle service management involves multiple activities including vehicle registration, service booking, maintenance history, invoices, service-center coordination, notifications and emergency roadside assistance. When these activities are handled through disconnected channels, vehicle owners may spend additional time communicating with service providers and maintaining records.

DriveCare PRO is a centralized digital vehicle self-service platform developed to bring these activities into one application. The system provides registration and login, vehicle management, service appointment booking, digital service records, service-center information, notifications, roadside assistance and dashboard analytics. The application follows a layered architecture in which the frontend communicates with a Node.js and Express.js REST API. Business logic is separated into routes, controllers and middleware, while MongoDB stores application data through Mongoose models.

Authentication is implemented using JSON Web Tokens (JWT), and passwords are handled using bcryptjs hashing. The backend includes protected routes and role-based authorization for selected advisor and administrator operations. The project demonstrates the integration of frontend development, RESTful API design, document database modelling, authentication middleware and dashboard-oriented service workflows.

The resulting platform provides a structured digital workflow for vehicle owners and service-center operations, creating a foundation for future improvements such as predictive maintenance, live roadside tracking, online payments, mobile applications and advanced analytics.

Index

S. No.	Chapters	Topics	Page No.
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5–10
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	11–14
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version Control 3.5 APIs / External services	15–22
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	23–27
5	Design	5.1 Design principles 5.2 ER Diagram / Database schema 5.3 Design decisions and assumptions	28–32
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & Authorization	33–41
7	Features	7.1 List of main features 7.2 How each feature works	42–48
8	Testing	8.1 Postman testing 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	49–56
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	57–62
10	Challenges & Limitations	10.1 Issues faced 10.2 Solutions applied 10.3 Current limitations / known issues	63–68
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations / optimizations	69–72
12	Conclusion	12.1 Summary 12.2 What was achieved 12.3 Skills learned	73–76
13	References	Books, tutorials, APIs,	77–79

		documentation sites	
14	Appendices	Screenshots, code snippets, installation/setup, API summary	80–85

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Automotive service management is becoming increasingly dependent on digital systems because vehicle owners expect convenient access to service information, appointment scheduling, maintenance history and emergency support. In a conventional workflow, service information may be distributed across phone calls, paper invoices, workshop records and separate communication channels. This makes it difficult for a vehicle owner to obtain a complete and consistent view of the vehicle's service history.

The DriveCare PRO project was conceived as a digital self-service platform that brings the major vehicle-service activities into a unified environment. Instead of treating vehicle information, appointments, service records and roadside assistance as separate processes, the system connects them through a common user account and backend API.

From a software-engineering perspective, the project combines a browser-based frontend with a Node.js and Express.js backend and a MongoDB database accessed through Mongoose. This separation allows the interface, business logic and data management responsibilities to be maintained independently. REST APIs form the communication boundary between the frontend and backend.

The project also introduces authenticated access through JWT. After login, a token is used to identify the user on protected requests. Middleware verifies the token before selected controllers are allowed to access protected operations. This provides a clear security boundary between public and authenticated functionality.

1.2 Problem Statement

Traditional vehicle service management often depends on manual communication, physical visits and fragmented records. Service booking may require phone calls or direct workshop visits, while maintenance history, invoices and warranty information can be difficult to retrieve from one place. Emergency roadside support may also require a separate communication process. Customers additionally need timely information about appointments and service status.

The core problem addressed by DriveCare PRO is therefore the absence of a centralized digital workflow that connects vehicle owners with vehicle information, service appointments, service records, service centers, notifications and roadside assistance. The project aims to reduce fragmentation by placing these functions behind one application and one REST-based backend.

1.3 Scope of the Project

- User registration, login and profile management.

- Vehicle registration, viewing, updating and deletion.
- Service-center discovery and service-center details.
- Appointment creation, viewing and status management.
- Digital service records and invoice/payment information.
- Roadside assistance request creation and authorized status updates.
- User notifications and read/unread handling.
- Customer dashboard and advisor/admin analytics.
- Role-aware access for customer, advisor and administrator users.

1.4 Objective(s)

1. Develop a centralized digital platform for vehicle owners to manage vehicle and service information.
2. Provide an online service appointment workflow with service-center and service-type information.
3. Maintain digital service records and invoice-related information.
4. Provide a dedicated roadside assistance request workflow.
5. Provide notifications and dashboard information for service activities.
6. Implement authenticated and role-aware REST API access.

1.5 Importance of the Application

The importance of DriveCare PRO lies in its ability to reduce fragmentation in vehicle servicing. A centralized digital record can make service information easier to access, while an appointment workflow can reduce dependence on manual scheduling. A roadside assistance module provides a structured mechanism for emergency requests, and notifications can keep users informed of service-related events.

1.6 Target Users / Audience

User	Purpose in the system
Customer / Vehicle Owner	Manage vehicles, appointments, service records, notifications and roadside requests.
Advisor / Service Center	Perform authorized service-related operations and update selected records or requests.
Administrator	Perform authorized system-level and analytical operations.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

DriveCare PRO is a software-oriented web application and does not require specialized hardware for its core implementation. Development can be performed on a standard laptop or desktop capable of running Node.js, a modern browser, MongoDB connectivity and an IDE.

Component	Recommended requirement
Processor	Modern dual-core or better processor.
Memory	4 GB RAM or more for comfortable development.
Storage	Sufficient free space for Node modules, source code and database tools.
Network	Internet connection when using remote MongoDB or online development services.
Display	Desktop/laptop display suitable for browser-based application testing.

2.2 Software Requirements

- Node.js and npm.
- MongoDB local installation or a MongoDB-compatible connection string.
- Modern browser such as Chrome, Edge or Firefox.
- Visual Studio Code or another JavaScript development environment.
- Git and GitHub for version control.
- Postman for REST API testing.

2.3 Development Tools and Frameworks

Layer	Technology / Tool	Purpose
Frontend	HTML, CSS, JavaScript, Vite	User interface and frontend development.
Backend	Node.js, Express.js	REST API server and application logic.
Database	MongoDB	Persistent document storage.
ODM	Mongoose	Models, schema rules, references and database access.
Authentication	jsonwebtoken	JWT creation and verification.
Password Security	bcryptjs	Password hashing and comparison.
Middleware	cors, morgan, express	Cross-origin handling, logging and request parsing.
Configuration	dotenv	Environment configuration.
Testing	Postman	API request and response testing.
Version Control	Git, GitHub	Source control and collaboration.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End

The frontend provides the user-facing portion of DriveCare PRO. The project includes a Vite-based frontend package and a browser interface organized around the vehicle-service workflow. The interface contains areas for dashboard information, vehicles, appointments, service records, roadside assistance, service centers, notifications and profile or advisor functions.

The interface is designed around a modern self-service experience. The supplied project screenshots demonstrate a login interface and a dashboard containing active-fleet information, upcoming services, health information and service actions. The frontend communicates with backend services through HTTP requests and JSON responses.

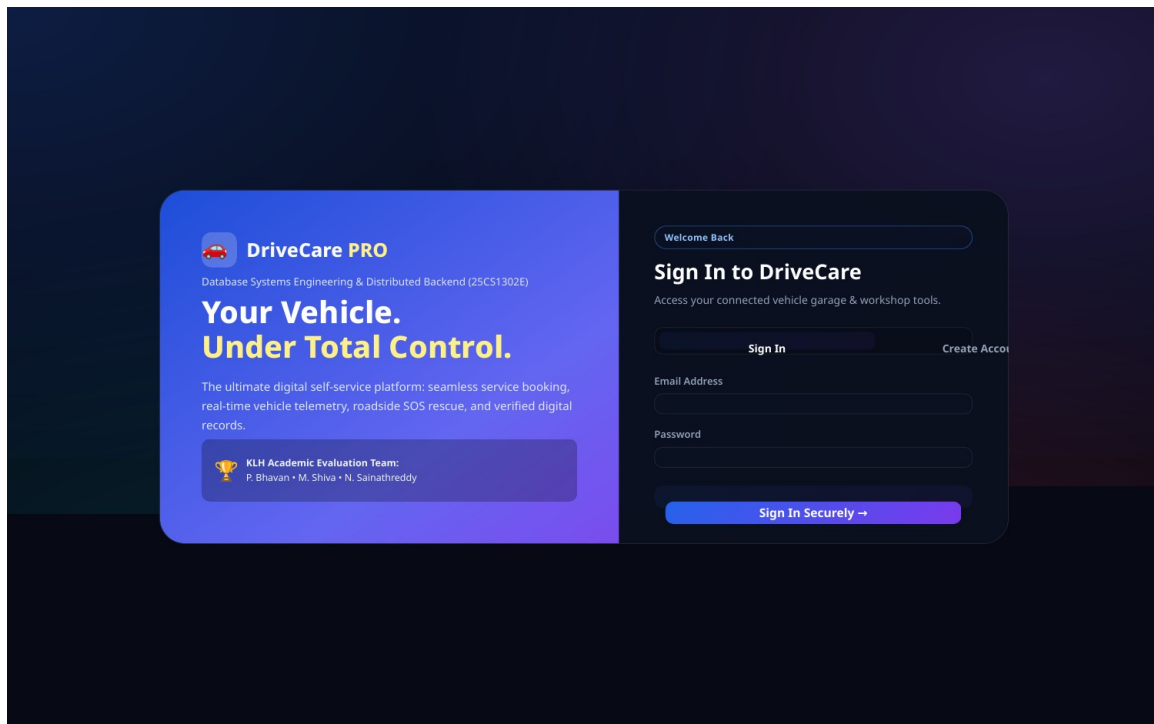


Figure 3.1 – DriveCare PRO login interface

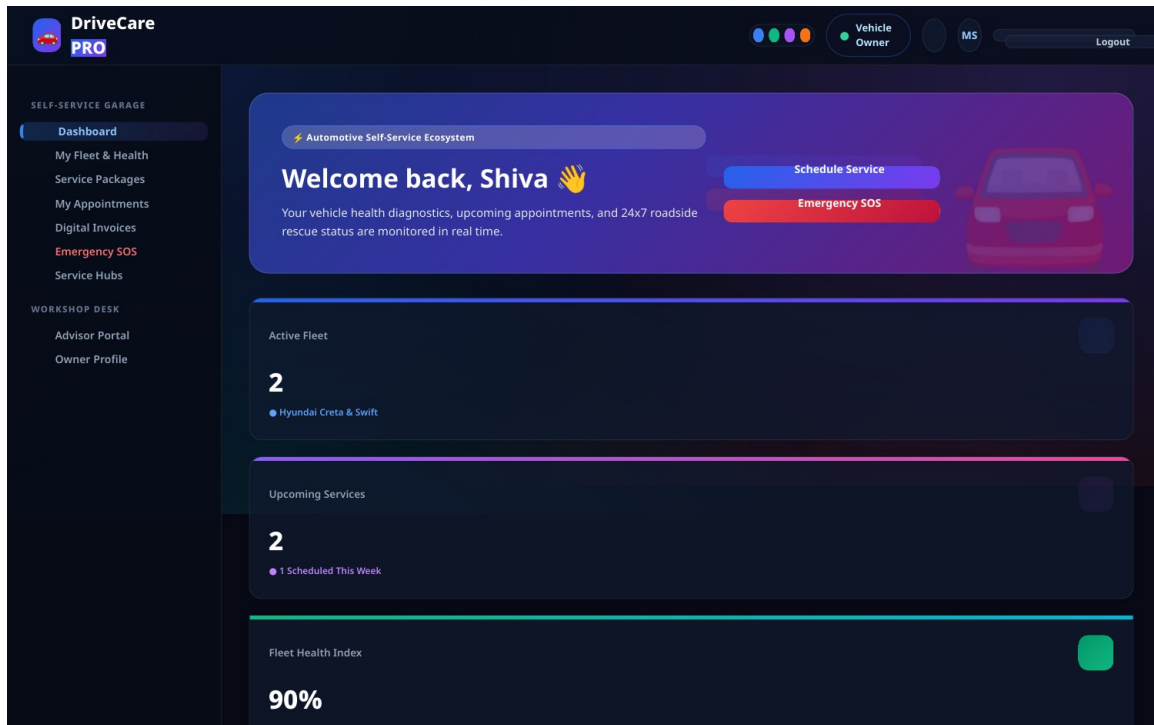


Figure 3.2 – DriveCare PRO dashboard

3.2 Back-End

The backend is implemented using Node.js and Express.js. Express provides the routing and middleware framework through which API requests are received, authenticated and passed to controllers. The server configures CORS, JSON parsing, URL-encoded request parsing, request logging, static frontend delivery and centralized error handling.

The backend follows a modular organization using routes, controllers, middleware and models. Each major application resource has a corresponding route/controller structure. This separation keeps request routing distinct from business processing and data-access definitions.

```
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/vehicles', require('./routes/vehicleRoutes'));
app.use('/api/appointments', require('./routes/appointmentRoutes'));
app.use('/api/service-records', require('./routes/serviceRecordRoutes'));
app.use('/api/roadside', require('./routes/roadsideRoutes'));
app.use('/api/service-centers', require('./routes/serviceCenterRoutes'));
app.use('/api/notifications', require('./routes/notificationRoutes'));
app.use('/api/analytics', require('./routes/analyticsRoutes'));
```

3.3 Database

MongoDB is used as the persistent database and Mongoose is used as the object-document modelling layer. The project defines separate models for User, Vehicle, Appointment, ServiceRecord, ServiceCenter, Notification and RoadsideRequest. Relationships are represented through MongoDB ObjectId references.

3.4 Version Control

Git and GitHub are used as the project's source-control and collaboration tools. Version control provides a mechanism for maintaining project history, coordinating changes and sharing the implementation.

3.5 APIs or External Services

The main application communication is provided through internal REST APIs exposed by the Express backend. Postman is used as an API development and testing tool. The submitted implementation also contains configuration support for MongoDB connection details and environment variables. No unsupported external service is claimed as a core dependency of the current implementation.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

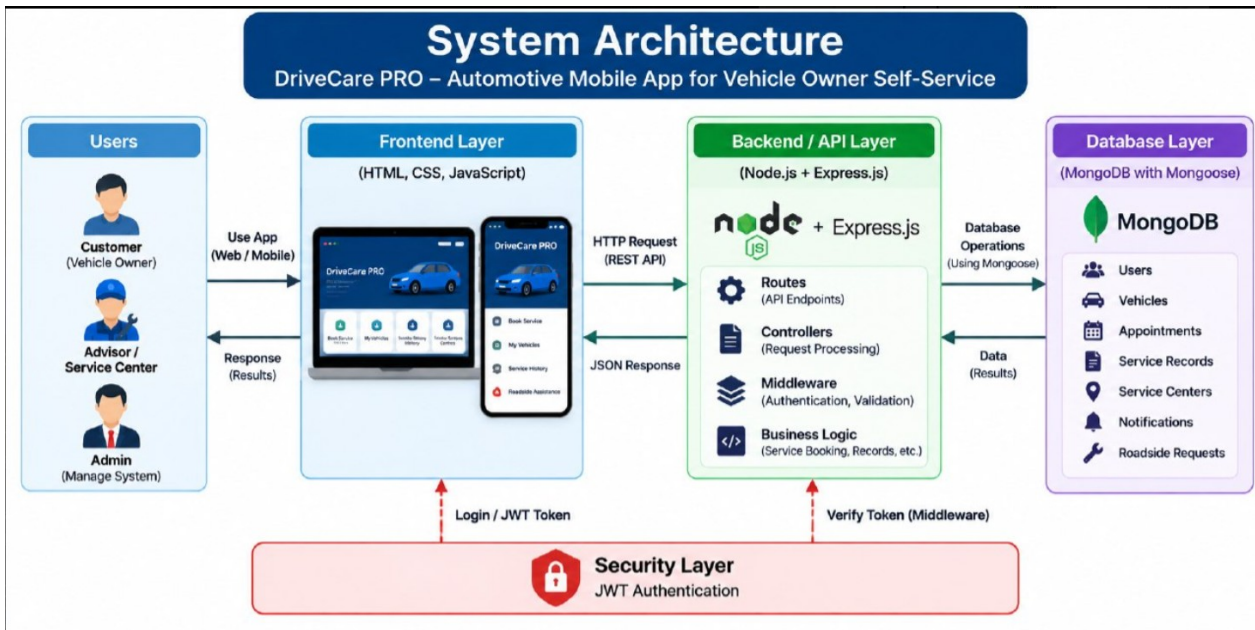


Figure 4.1 – DriveCare PRO high-level layered architecture

The architecture separates the system into users, frontend, backend/API, database and authentication concerns. Customers, advisors/service centers and administrators interact with the DriveCare PRO interface. The frontend sends HTTP requests through REST APIs to the Node.js and Express.js backend. The backend processes requests through routes, controllers and middleware and accesses MongoDB through Mongoose.

4.2 Description of Each Layer

The Frontend Layer is responsible for presenting the user interface and collecting user actions. It displays vehicle information, service options, appointments, records and assistance functions.

The Backend/API Layer receives REST requests and returns JSON responses. Express routes determine the endpoint, middleware performs authentication and validation-related checks, and controllers perform the application operations.

The Business Layer contains request processing and service-management logic. It is separated from the route definitions so that the backend remains modular and easier to maintain.

The Database Layer uses MongoDB with Mongoose. User, vehicle, appointment, service record, service center, notification and roadside request data are persisted as documents.

The Authentication Layer uses JWT. After login, the token is included in the Authorization header for protected requests. Middleware verifies the token before allowing protected controllers to continue.

4.3 Deployment Architecture

The current project is suitable for local development using Node.js, npm and MongoDB or a MongoDB-compatible remote database. The source contains environment-based configuration for the database connection, JWT secret and server port.

For a production deployment, the frontend and backend can be hosted using appropriate web hosting or cloud infrastructure, while MongoDB can be hosted through a managed MongoDB service. HTTPS, secure environment variables and restricted CORS should be enabled before public deployment. These are deployment recommendations rather than claims that every listed production service is already configured in the submitted project.

CHAPTER -5 DESIGN

5.1 Database Design Principles

The database design separates major business entities into independent Mongoose models. This prevents unrelated information from being stored in a single large document and makes individual operations easier to implement. Relationships are represented using ObjectId references so that related records can be retrieved when required.

The models also use required fields, enum values, indexes, unique constraints where applicable, timestamps and validation rules. These constraints help maintain consistency between the application layer and the stored data.

5.2 ER Diagram / Database Schema

Entity	Purpose	Key Relationships
USER	Stores account and profile information.	Owns vehicles; receives notifications; creates appointments and roadside requests.
VEHICLE	Stores vehicle identity and health/service information.	Belongs to USER; used by appointments, records and roadside requests.
APPOINTMENT	Stores scheduled service activity.	Links USER, VEHICLE and SERVICE CENTER.
SERVICE RECORD	Stores completed service and invoice information.	Links USER, VEHICLE, APPOINTMENT and SERVICE CENTER.
SERVICE CENTER	Stores service-facility information and offered services.	Referenced by appointments and service records.
NOTIFICATION	Stores user notifications.	Belongs to USER.
ROADSIDE REQUEST	Stores emergency assistance requests.	Links USER, VEHICLE and service/assistance information.

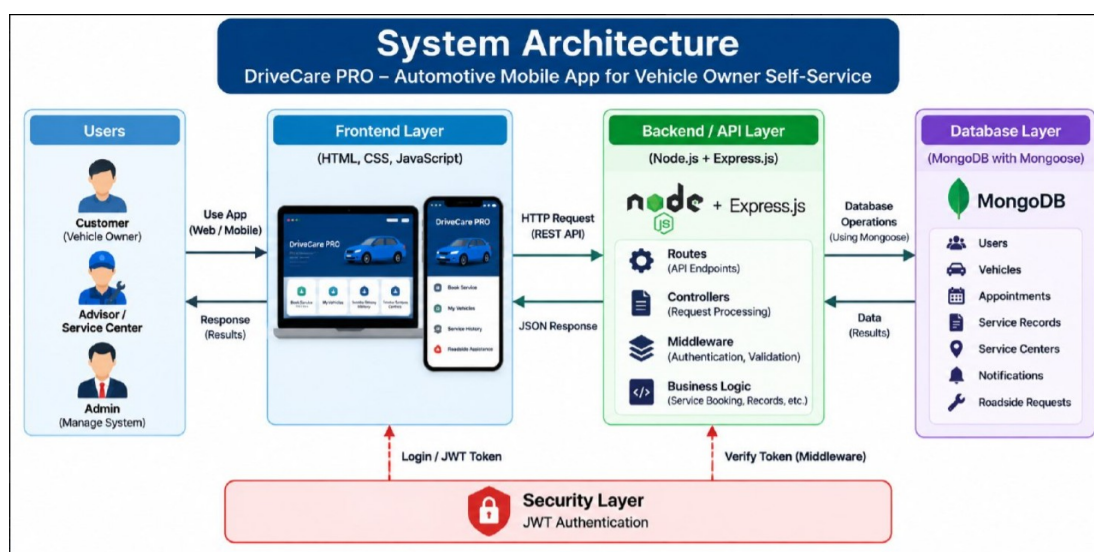


Figure 5.1 – Architecture reference showing database entities and system relationships

5.3 Design Decisions and Assumptions

A document-oriented MongoDB model was selected because the application contains several related but flexible service records, health information, notifications and emergency-request details. Mongoose provides a structured model layer over these documents.

The User model uses a role field to distinguish customer, advisor and admin users. Authentication is separated from role authorization: the JWT establishes the authenticated identity, while authorization middleware determines whether the user's role is permitted to perform a selected operation.

Service-center location data is represented in the model using location information suitable for geospatial indexing. This design provides a foundation for nearby-service-center queries.

The project treats the submitted implementation as the source of truth. Features such as live payments, live GPS tracking and AI prediction are documented as future enhancements rather than current implemented features.

CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The implementation of DriveCare PRO is organized into resource modules. Each module has a defined responsibility and communicates with the database through Mongoose models. The modular structure allows authentication, vehicle operations, appointments, service records, roadside requests, service centers, notifications and analytics to evolve independently while sharing the same backend.

Module	Route Base	Main Operations
Authentication	/api/auth	Register, login, current user and profile update.
Vehicles	/api/vehicles	List, create, read, update and delete vehicles.
Appointments	/api/appointments	List, create, read and update appointment status.
Service Records	/api/service-records	List, read, create by authorized roles and invoice payment operation.
Roadside	/api/roadside	Create/list requests and authorized status updates.
Service Centers	/api/service-centers	List service centers and read individual centers.
Notifications	/api/notifications	List, mark one read and mark all read.
Analytics	/api/analytics	Customer dashboard and advisor/admin analytics.

6.2 Front-End Logic (UI Rendering and State)

The frontend presents the DriveCare PRO self-service workflow through a browser interface. The dashboard exposes high-level vehicle and service information, while navigation provides access to vehicles, appointments, service records, roadside assistance, service centers and notifications. The login interface collects email and password credentials and initiates the authentication workflow.

The UI communicates with backend endpoints rather than directly accessing MongoDB. This preserves the separation between presentation and data access. Backend responses are returned in JSON and are used to update the interface.

6.3 API Integration

API integration is based on REST endpoints exposed by Express.js. Authentication endpoints handle registration and login, while resource endpoints handle vehicle, appointment, service-record, roadside, service-center, notification and analytics operations.

```
GET    /api/vehicles
POST   /api/vehicles
GET    /api/vehicles/:id
PUT    /api/vehicles/:id
DELETE /api/vehicles/:id
```

```
GET    /api/appointments
POST   /api/appointments
PATCH /api/appointments/:id/status
```

Protected requests carry a JWT in the Authorization header. The backend verifies the token before executing protected operations.

6.4 Authentication & Authorization

The authentication controller supports registration and login. During registration, the password is hashed using bcryptjs before it is stored. During login, the entered password is compared with the stored hash and a JWT is generated after successful authentication.

```
Authorization: Bearer <JWT_TOKEN>
```

The protect middleware verifies the JWT and loads the authenticated user. The authorize middleware is then used on selected operations to restrict access to advisor or admin roles. This provides two separate checks: identity verification and permission verification.

CHAPTER 7- FEATURES

7.1 List of Main Features

- User registration and login.
- JWT-based authenticated access.
- Vehicle management and vehicle-health information.
- Service-center discovery and service-center information.
- Service appointment booking and status tracking.
- Digital service records and invoice/payment information.
- Notifications and read/unread management.
- Roadside assistance requests.
- Customer dashboard analytics.
- Advisor/admin analytics and authorized service operations.

7.2 How Each Feature Works (User Perspective + Technical Description)

1. Authentication

From the user perspective, registration creates an account and login provides access to the application. Technically, the backend verifies credentials, hashes passwords through bcryptjs and returns a JWT. Protected requests include the token in the Authorization header.

2. Vehicle Management

Users can maintain their vehicle information. The backend provides CRUD operations through the vehicle routes and stores the information using the Vehicle Mongoose model.

3. Appointment Booking

The user selects a vehicle, service center, service type and scheduled date/time. The appointment is stored with a status that can progress through the defined service workflow. The backend exposes endpoints for appointment creation, retrieval and status updates.

4. Service Records

Service records provide a digital history of work performed on a vehicle. The model stores service details, invoice information, payment status and related references.

5. Service Centers

Service-center endpoints provide access to service-center information. The ServiceCenter model stores facility details, contact information, location and offered services.

6. Notifications

Notifications are linked to users and include a title, message, type, read status and optional link. Users can retrieve notifications and mark them as read.

7. Roadside Assistance

Users can submit a roadside request with emergency type, urgency, contact and location information. Authorized advisor/admin operations can update the request status.

8. Analytics

Dashboard analytics aggregate vehicle count, health information, appointments, spending, open roadside requests and related service metrics. Advisor/admin analytics provide operational summaries such as appointment status and revenue by service type.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman provides a practical environment for validating the REST interface independently of the graphical frontend. For DriveCare PRO, the testing workflow can be organized around authentication, CRUD resources, protected routes, role restrictions and response validation.

Authentication requests can be tested first so that a valid JWT is obtained. The token can then be added to the Authorization header for vehicle, appointment, service-record, roadside and notification requests. Both successful and invalid requests should be exercised to verify expected status codes and error messages.

Area	Example Request	Verification
Registration	POST /api/auth/register	New account created; duplicate email rejected.
Login	POST /api/auth/login	Valid credentials return token; invalid credentials rejected.
Protected access	GET /api/auth/me	Token required and user returned.
Vehicle CRUD	POST/GET/PUT/DELETE /api/vehicles	Create, read, update and delete behavior.
Appointments	POST /api/appointments	Required booking information validated.
Status update	PATCH /api/appointments/:id/status	Status changes according to authorized operation.
Roadside	POST /api/roadside	Request created with emergency details.
Notifications	PATCH /api/notifications/:id/read	Notification read state updated.

8.2 Integration Testing

Integration testing verifies that the frontend, backend, authentication middleware and MongoDB layer work together. A typical workflow begins with login, continues with vehicle retrieval, creates an appointment, reads service data and then displays the resulting information in the frontend.

The database integration is verified by confirming that successful API operations create or update the expected MongoDB documents and that subsequent GET requests return the stored information. Authentication integration is verified by checking that protected requests fail without a valid token and succeed when a valid token is supplied.

8.3 Tools Used for Testing

- Postman for REST API requests and response inspection.
- Browser testing for login, dashboard and application workflows.
- MongoDB/Mongoose inspection for stored data verification.
- VS Code terminal for server logs and development diagnostics.

8.4 Test Cases and Results

TC ID	Test Case	Expected Result	Status
TC01	Register new user	Account created and token returned.	Pass / verify in environment
TC02	Register duplicate email	Duplicate account rejected.	Pass / verify in environment
TC03	Login valid credentials	JWT returned.	Pass / verify in environment
TC04	Login invalid credentials	401 authentication error.	Pass / verify in environment
TC05	Protected API without token	Request rejected.	Pass / verify in environment
TC06	Vehicle CRUD	Vehicle data persists correctly.	Pass / verify in environment
TC07	Create appointment	Appointment stored with status.	Pass / verify in environment
TC08	Roadside request	Request stored successfully.	Pass / verify in environment
TC09	Unauthorized role operation	403 authorization error.	Pass / verify in environment
TC10	Notification read update	isRead changes to true.	Pass / verify in environment

The submitted source contains the corresponding routes, controllers, middleware and models for these scenarios. Because a formal automated test suite is not present in the source documentation available for this report, the results are presented as a structured verification matrix rather than as a claim of automated test execution.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Deployment of DriveCare PRO begins with preparing the Node.js backend, frontend and MongoDB connection. The source is organized into backend and frontend packages, allowing the components to be installed and run independently during development.

7. Install Node.js and npm on the development or deployment machine.
8. Install backend dependencies using npm install inside the backend directory.
9. Configure the MongoDB connection string and JWT secret through environment variables.
10. Start the Express backend and verify the /api/health endpoint.
11. Install frontend dependencies and start the Vite development server.
12. Configure the frontend to communicate with the backend API.
13. Test login and protected endpoints before exposing the application to users.
14. For production, build and host the frontend and backend using secure deployment infrastructure.

9.2 Environment Configuration

```
MONGO_URI=<MongoDB connection string>  
JWT_SECRET=<strong secret value>  
PORT=5000
```

The application reads the MongoDB connection string and JWT secret from environment configuration. Secrets should not be hard-coded in a public repository. The backend also uses a configurable port.

9.3 Hosting of Frontend and Backend

The project can be deployed with the frontend and backend hosted separately or together depending on the final demonstration environment. A managed MongoDB service can be used for remote persistence. For a production environment, HTTPS, secure secrets, restricted CORS and monitoring should be configured.

The exact hosting provider is not treated as an implemented project feature unless configured in the submitted environment. The deployment section therefore documents the supported deployment approach and recommended production configuration without claiming an unverified cloud deployment.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Maintaining consistent authentication across many protected REST resources.
- Keeping references between users, vehicles, appointments and service records consistent.
- Separating customer functions from advisor/admin operations.
- Maintaining a modular Express route/controller/model structure.
- Keeping frontend workflows synchronized with backend JSON responses and service statuses.
- Combining data from multiple MongoDB collections for dashboard analytics.

10.2 Solutions Applied

Authentication was separated into reusable middleware so that protected routes can share the same JWT verification mechanism. Role authorization was implemented separately so that selected operations can restrict access to advisor or administrator users.

Mongoose models were used to define entity structures and references. Separate route and controller modules keep endpoint definitions and request-processing logic organized. MongoDB indexes and schema constraints support efficient and consistent data access.

The backend also uses centralized error handling and request logging. This gives the development workflow a consistent place to report unexpected server errors and inspect request activity.

10.3 Current Limitations or Known Issues

- The current project does not demonstrate a complete automated unit/integration test suite.
- Service-center functionality primarily exposes read operations in the current API structure; full administrative CRUD can be added later.
- Production hosting configuration is not treated as part of the current implementation unless separately configured.
- Live GPS roadside tracking is not part of the current core implementation.
- AI-based predictive maintenance is a future enhancement rather than a current implemented model.
- Online payment gateway integration is a future enhancement.
- Push-notification infrastructure beyond the current notification data model can be expanded later.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The current project provides the foundation for several extensions. Future development can add intelligence and automation without changing the fundamental separation between frontend, REST API, business logic and MongoDB.

- AI-based predictive maintenance using service history, mileage and health information.
- Live roadside-assistance tracking using location updates.
- Online payment gateway integration for service invoices.
- Dedicated Android/iOS mobile application using Flutter.
- Advanced analytics for service-center performance and vehicle-service trends.
- Automated service reminders based on mileage and service dates.
- Push, email and SMS notifications for appointments and emergency events.

11.2 Possible Integrations or Optimizations

The ServiceCenter model can be extended with richer geospatial search and availability information. Notification delivery can be connected to a push messaging service. Analytics can be expanded with time-series service trends and maintenance-cost summaries. Automated API testing can be introduced using a Postman collection or a JavaScript testing framework.

For production optimization, database indexes can be reviewed using real query patterns, API payloads can be minimized, and frontend assets can be optimized. A managed MongoDB deployment, HTTPS and centralized monitoring can improve reliability when the application is exposed to real users.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

DriveCare PRO is a centralized vehicle-service self-service platform that combines user management, vehicle information, service appointments, service records, service-center information, notifications, roadside assistance and analytics. The project follows a layered architecture using a frontend interface, Node.js and Express.js REST APIs, business controllers and a MongoDB database accessed through Mongoose.

The system addresses the fragmentation that can occur when service booking, maintenance records and emergency support are handled through separate channels. By bringing these functions into one application, DriveCare PRO provides a structured digital workflow for vehicle owners and service-center operations.

12.2 What Was Achieved

- A working full-stack project structure was established.
- REST APIs were organized into authentication, vehicle, appointment, service-record, roadside, service-center, notification and analytics resources.
- MongoDB/Mongoose models were defined for the major application entities.
- JWT authentication and role-aware authorization were implemented for protected operations.
- Vehicle, appointment, service-record and roadside workflows were represented in the backend.
- Dashboard analytics were included for customer and advisor/admin views.
- A modern DriveCare PRO frontend/login/dashboard interface was developed.

12.3 Skills Learned During Development

- REST API design using Node.js and Express.js.
- MongoDB document modelling using Mongoose.
- Authentication and authorization using JWT middleware.
- Password hashing using bcryptjs.
- CRUD implementation and request validation.
- Frontend-to-backend integration using HTTP/JSON.
- Git/GitHub-based source management.
- API testing and debugging using Postman.
- Software architecture, database design and technical documentation.

Overall, DriveCare PRO demonstrates the practical application of database management, backend development, web interface design, authentication, API integration and software-engineering principles in an automotive service domain.

CHAPTER -13 REFERENCES

Books

Pressman, R. S., & Maxim, B. R. Software Engineering: A Practitioner's Approach.

Sommerville, I. Software Engineering.

Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems.

Tutorials and Learning Resources

MDN Web Docs. HTML, CSS and JavaScript documentation.

Node.js Documentation.

Express.js Documentation.

MongoDB Documentation.

Mongoose Documentation.

Git Documentation.

GitHub Documentation.

Postman Learning Center.

APIs and Tools Used

Node.js – server runtime environment.

Express.js – REST API framework.

MongoDB – document database.

Mongoose – MongoDB object-document modelling library.

jsonwebtoken – JSON Web Token implementation.

bcryptjs – password hashing library.

Postman – API testing tool.

Vite – frontend development/build tool.

Project Documentation and Research References

DriveCare PRO submitted source code, presentation material, architecture diagram and project interface screenshots.

KLH/FEDF project documentation format supplied as the structural reference for this report. The reference format contains chapters for introduction, system requirements, technology stack, architecture, design, implementation, features, testing, deployment, challenges and limitations, future enhancements, conclusion, references and appendices.

CHAPTER -14 APPENDICES

Appendix A – Screenshots of the Application

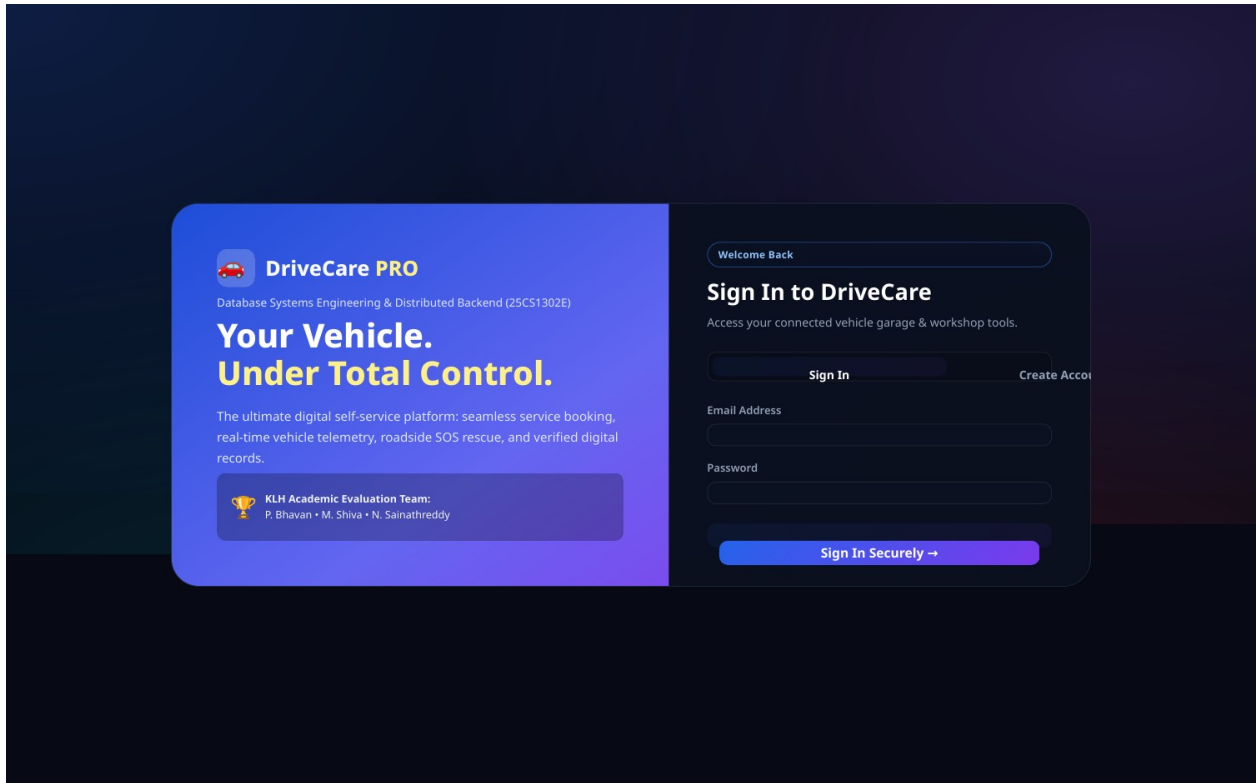


Figure A.1 – DriveCare PRO login screen

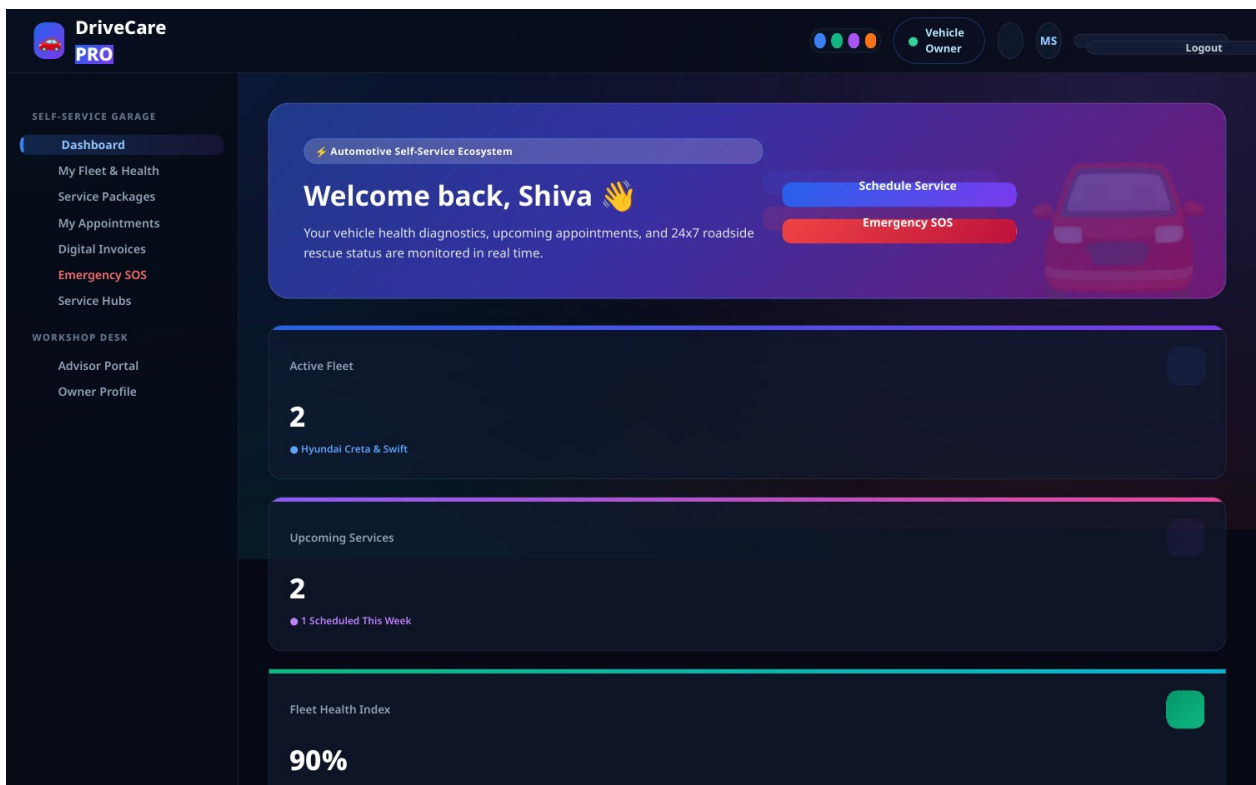


Figure A.2 – DriveCare PRO dashboard

Appendix B – System Architecture

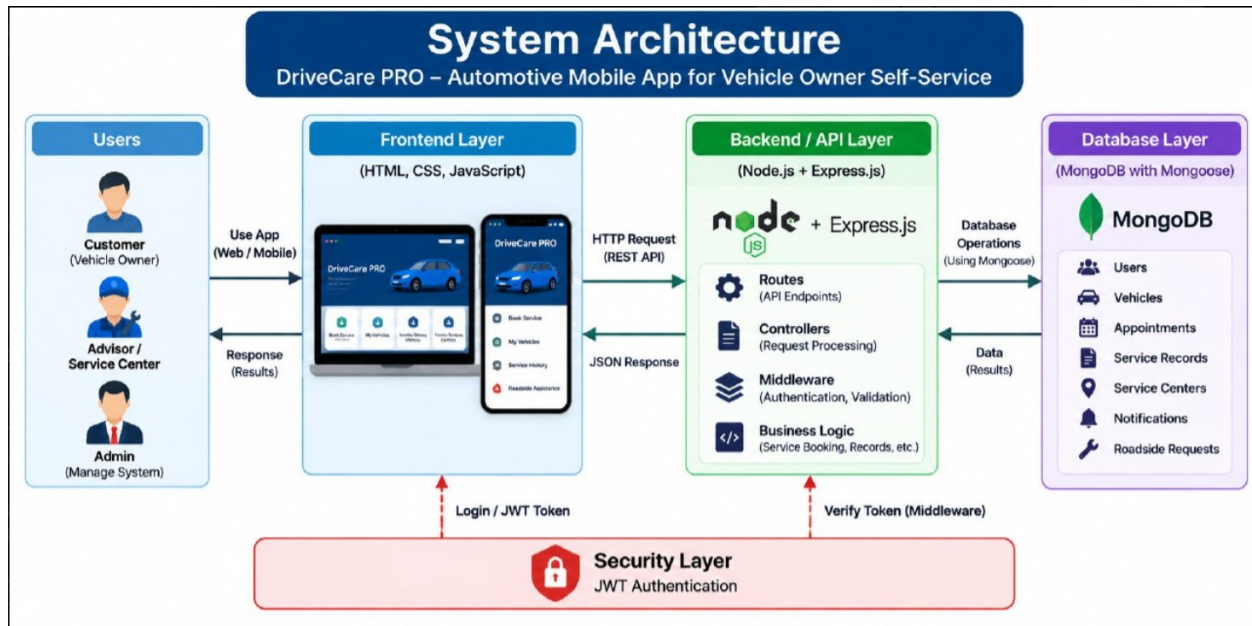


Figure B.1 – DriveCare PRO system architecture

Appendix C – Sample Code Snippets

C.1 MongoDB Connection

```
const mongoose = require('mongoose');

const connectDB = async () => {
  const conn = await mongoose.connect(
    process.env.MONGO_URI ||
    'mongodb://127.0.0.1:27017/drivecare_db'
  );
  console.log(
    `[MongoDB Connected] Host: ${conn.connection.host}, ` +
    `Database: ${conn.connection.name}`
  );
};
```

C.2 JWT Verification

```
const decoded = jwt.verify(
  token,
  process.env.JWT_SECRET
);

req.user = await User.findById(decoded.id)
  .select('-password');
```

C.3 Role Authorization

```
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: 'User role is not authorized'
      });
    }
  };
};
```

```
        next();  
    };  
};
```

Appendix D – Installation / Setup Instructions

D.1 Backend

```
cd backend
npm install
npm start

# Development
npm run dev
```

D.2 Environment

```
MONGO_URI=<your MongoDB URI>
JWT_SECRET=<your JWT secret>
PORT=5000
```

D.3 Frontend

```
cd frontend
npm install
npm run dev
```

D.4 Basic Verification

15. Start MongoDB or configure a remote MongoDB URI.
16. Start the backend server.
17. Check GET /api/health.
18. Start the frontend.
19. Open the DriveCare PRO login page.
20. Register or use a valid account.
21. Verify vehicle, appointment and dashboard workflows.
22. Use Postman to verify protected REST endpoints.

Appendix E – API Summary

Resource	Methods / Endpoints
Authentication	POST /api/auth/register POST /api/auth/login GET /api/auth/me PUT /api/auth/profile
Vehicles	GET /api/vehicles POST /api/vehicles GET /api/vehicles/:id PUT /api/vehicles/:id DELETE /api/vehicles/:id
Appointments	GET /api/appointments POST /api/appointments GET /api/appointments/:id PATCH /api/appointments/:id/status
Service Records	GET /api/service-records POST /api/service-records GET /api/service-records/:id POST /api/service-records/:id/pay
Roadside	GET /api/roadside POST /api/roadside PATCH /api/roadside/:id/status
Service Centers	GET /api/service-centers

	GET /api/service-centers/:id
Notifications	GET /api/notifications PATCH /api/notifications/:id/read POST /api/notifications/mark-all-read
Analytics	GET /api/analytics/dashboard GET /api/analytics/advisor

DRIVECARE PRO

Automotive Mobile App for Vehicle Owner Self-Service

PROJECT DOCUMENTATION | 2026–2027